CyberAI Homelab

AI-Powered Cybersecurity: Practical Implementation Project

Docker-based network security lab: Zeek, Suricata IDS, ELK Stack (Logstash, Elasticsearch, Kibana), Jupyter Notebooks (machine learning) and MISP (threat intelligence), running in Kali Linux on WSL2.

Item	Detail
Course	EduSkills - AI-Powered Cybersecurity, Module 1: Practical implementation (basic classification and clustering)
Machine	HP OmniBook 5, Windows 11, Snapdragon (ARM64) processor
Environment	WSL2 with Kali Linux, Docker Engine inside Kali (not Docker Desktop)
WSL resources	10 GB RAM, 4 processors, 4 GB swap (set in .wslconfig)
Lab folder	~/cyberai-lab (inside Kali)
Document date	02 October 2026

Safety note: Capture and analyze traffic only on networks you own or have permission to monitor. Run the lab for learning only; security is intentionally disabled on Elasticsearch, so never expose it to the internet.

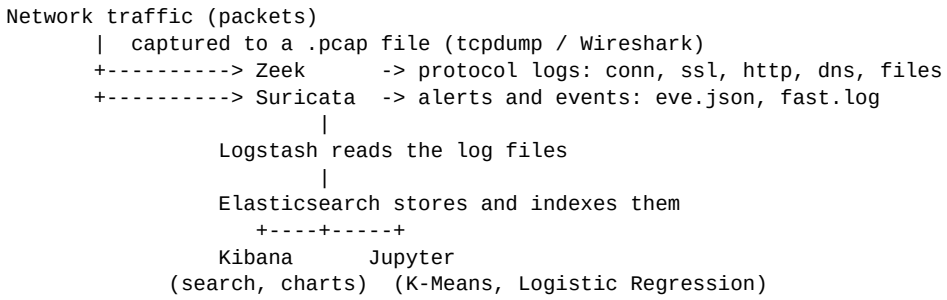
1. Project overview

The goal is to build a local security-analytics lab and use it to apply basic machine-learning methods to network data. Network traffic is captured, analyzed by two sensors (Zeek and Suricata), shipped through Logstash into Elasticsearch, explored in Kibana, and analyzed with Python in Jupyter.

Lesson learning objectives (from the course page):

- Set up and configure the CyberAI homelab environment.
- Install and manage Python and key machine-learning libraries.
- Configure network analysis tools (Zeek, Suricata).
- Set up the ELK Stack for data visualization.
- Configure MISP for threat-intelligence sharing.
- Use Jupyter Notebooks for interactive analysis.
- Implement a Logistic Regression model (binary classification).
- Implement a K-Means clustering model (unsupervised learning).
- Evaluate the models with basic metrics.

2. Architecture and how it works



Tool	Role	Port
Docker	Runs ELK and Jupyter (and MISP) as containers on one network	-
Zeek	Network analyzer. Describes what happened (who talked to whom, how much, which protocol). Does not judge.	native in Kali
Suricata	IDS/IPS. Compares traffic with about 53,000 rules (Emerging Threats) plus custom rules and raises alerts.	native in Kali
Logstash	Reads Zeek and Suricata log files, cleans fields, sends to Elasticsearch.	-
Elasticsearch 8.14.0	Stores and indexes events. Index name pattern: cyberai-YYYY.MM.DD	9200
Kibana 8.14.0	Web GUI for searching, charts and dashboards.	5601
Jupyter (scipy-notebook)	Python notebooks for machine learning. Reaches Elasticsearch at elasticsearch:9200.	8888
MISP	Threat-intelligence platform (indicators of compromise). Separate compose project.	443 (https)

3. Project status: what is done and what is left

Component	Status	Evidence / note
WSL2 + Kali Linux	DONE	wsl -d kali-linux works; 10 GB RAM via .wslconfig
Docker Engine in Kali	DONE	docker run hello-world succeeded (arm64v8 image)
Zeek 9.0.0	DONE	Produces JSON logs; PATH fixed (/opt/zeek/bin)
Suricata 8.0.6	DONE	69,025 rules loaded; custom rule sid 1000001 fired
Elasticsearch / Kibana / Logstash	DONE	Cluster green, 63 documents, 0 indexing errors
Jupyter + K-Means	DONE	20 connections clustered into 3 groups, verified
Kibana data view + dashboard	TO DO	GUI step: create data view cyberai-* and a dashboard
Live Suricata to Kibana	IN PROGRESS	Started with -l ~/cyberai-lab/logs/suricata; confirm alerts reach Kibana
Logistic Regression	TO DO	Needs a labeled dataset (NSL-KDD / CIC-IDS2017 / course file)
MISP	TO DO	misp-core was unhealthy when RAM was short; retry alone with ELK stopped
Evaluation + report	TO DO	Metrics, screenshots, written conclusions

4. All project files and where they are

Location	File / folder	Purpose
Windows: C:\Users\PRATIK BHOSLE\	.wslconfig	Limits WSL to memory=10GB, processors=4, swap=4GB
Kali: ~/cyberai-lab/	docker-compose.yml	Defines elasticsearch, kibana, logstash, jupyter and volume esdata
	logstash/pipeline.conf	Inputs (Zeek + Suricata logs), filters, output to Elasticsearch
	local.rules	Custom Suricata rule (sid 1000001)
	pcaps/sample.pcap, sample2.pcap	Captured traffic (14 and 409 packets)
	logs/zeek/	Zeek output: conn, ssl, http, files, packet_filter
	logs/suricata/	eve.json, fast.log, stats.log, suricata.log
	notebooks/check.py	Counts documents, lists Suricata alert signatures
	notebooks/kmeans.py	K-Means clustering of Zeek connections
	notebooks/check_clusters.py	Prints each connection with its cluster
Kali: ~/misp-docker/	.env, docker-compose.yml	MISP stack (cloned from MISP/misp-docker)
Kali: /etc/suricata/	suricata.yaml	Suricata main configuration
Kali: /var/lib/suricata/rules/	suricata.rules	Rules merged by suricata-update

5. Setup procedure (reproducible)

5.1 Windows and WSL2

```
# PowerShell
wsl --install            # then restart (only if WSL not installed)
wsl --update
notepad $env:USERPROFILE\.wslconfig
```

```
[wsl2]
memory=10GB
processors=4
swap=4GB
```

Then run `wsl --shutdown` and start Kali with `wsl -d kali-linux`. On 8 GB laptops use memory=5GB.

5.2 Install tools in Kali

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y docker.io docker-compose git python3-pip suricata zeek
sudo usermod -aG docker $USER
sudo service docker start
echo 'export PATH=/opt/zeek/bin:$PATH' >> ~/.zshrc && source ~/.zshrc
docker run --rm hello-world
```

5.3 Free port 9200 (important)

Kali had a native Elasticsearch 7.12.1 and Wazuh services using port 9200. They were stopped and disabled so Docker Elasticsearch can bind it:

```
sudo systemctl stop elasticsearch wazuh-indexer wazuh-manager filebeat
sudo systemctl disable elasticsearch wazuh-indexer
sudo ss -ltnp | grep -E '9200|5601'      # must print nothing
```

5.4 Every session (start routine)

```
wsl -d kali-linux
sudo service docker start
sudo sysctl -w vm.max_map_count=262144    # needed by Elasticsearch, resets after wsl --shutdown
cd ~/cyberai-lab && docker compose up -d
docker compose ps
```

5.5 Lab folders

```
mkdir -p ~/cyberai-lab/{logs/zeek,logs/suricata,pcaps,notebooks,logstash}
```

6. Configuration files (final versions)

6.1 docker-compose.yml

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.14.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
    ports: ["9200:9200"]
    volumes: [esdata:/usr/share/elasticsearch/data]
  kibana:
    image: docker.elastic.co/kibana/kibana:8.14.0
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    ports: ["5601:5601"]
    depends_on: [elasticsearch]
  logstash:
    image: docker.elastic.co/logstash/logstash:8.14.0
    volumes:
      - ./logstash/pipeline.conf:/usr/share/logstash/pipeline/logstash.conf
      - ./logs:/logs:ro
    environment:
      - LS_JAVA_OPTS=-Xms512m -Xmx512m
    depends_on: [elasticsearch]
  jupyter:
    image: quay.io/jupyter/scipy-notebook:latest
    ports: ["8888:8888"]
    volumes: ["./notebooks:/home/jovyan/work"]
    depends_on: [elasticsearch]
volumes:
  esdata:
```

6.2 logstash/pipeline.conf (input lines wrapped here for printing)

```
input {
  file { path => "/logs/suricata/eve.json" codec => "json"
         start_position => "beginning" sincedb_path => "/dev/null"
         tags => ["suricata"] }
  file { path => "/logs/zeek/*.log" codec => "json"
         start_position => "beginning" sincedb_path => "/dev/null"
         tags => ["zeek"] }
}
filter {
  if "zeek" in [tags] {
    ruby { code => '
      h = event.get("host")
      if h.is_a?(String)
        event.set("http_host", h); event.remove("host")
      elsif h.is_a?(Hash)
        event.remove("host")
      end' }
    date { match => ["ts", "UNIX"] target => "@timestamp" }
  }
  if "suricata" in [tags] {
    date { match => ["timestamp", "ISO8601"] target => "@timestamp" }
  }
}
output {
  elasticsearch { hosts => ["http://elasticsearch:9200"]
                  index => "cyberai-%{+YYYY.MM.dd}" }
}
```

The ruby filter fixes a real error found during the project: Zeek's HTTP 'host' field (a string) clashed with the 'host' object that Logstash adds, so Elasticsearch rejected those events (document_parsing_exception).

6.3 local.rules (custom Suricata rule)

```
alert http any any -> any any (msg:"LAB test - HTTP request to example.com";
  http.host; content:"example.com"; sid:1000001; rev:1;)
```

This is one single line in local.rules; it is wrapped here only for printing.

Part	Meaning
alert	Action. 'drop' would block the packet in IPS mode.
http any any -> any any	Protocol, source IP/port, direction, destination IP/port
msg	Alert name shown in logs
http.host; content:	Look for that text in the HTTP Host header
sid / rev	Unique rule ID (use 1000000+ for your own) and revision

6.4 Elasticsearch index template (zero replicas so the single node stays green)

```
curl -X PUT 'localhost:9200/_index_template/cyberai' -H 'Content-Type: application/json' \
  -d '{"index_patterns":["cyberai-*"],"template":{"settings":{"number_of_replicas":0}}}'
```

7. Running an analysis (capture to dashboard)

7.1 Capture traffic

```
sudo -v
sudo timeout 40 tcpdump -i eth0 -w ~/cyberai-lab/pcaps/sample2.pcap &
sleep 3
for i in 1 2 3; do curl -s http://example.com > /dev/null; curl -s http://neverssl.com > /dev/null; done
wait
```

7.2 Analyze with Zeek and Suricata

```
sudo chown -R $USER ~/cyberai-lab
cd ~/cyberai-lab/logs/zeek
zeek -C -r ~/cyberai-lab/pcaps/sample2.pcap LogAscii::use_json=T
suricata -c /etc/suricata/suricata.yaml -k none -S ~/cyberai-lab/local.rules \
   -r ~/cyberai-lab/pcaps/sample2.pcap -l ~/cyberai-lab/logs/suricata
```

Use `-S` to load only your rules (clean output) or `-s` to add them to the normal rule set. `-k none` disables checksum checks; WSL's virtual network card caused false 'invalid checksum' alerts without it.

7.3 Ingest into Elasticsearch

```
curl -X DELETE 'localhost:9200/cyberai-2026.10.02'     # exact index name; wildcards are refused
docker compose restart logstash
sleep 60
curl -s 'localhost:9200/_cat/indices/cyberai-*?v'
docker compose logs --since 2m logstash | grep -c "Could not index"   # want 0
```

7.4 Live monitoring (IDS mode)

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0 -k none \
   -s ~/cyberai-lab/local.rules -l ~/cyberai-lab/logs/suricata
# second terminal:  tail -f ~/cyberai-lab/logs/suricata/fast.log
# Ctrl+C to stop
```

Live mode only sees traffic inside WSL (Kali and the Docker containers), not Windows browser traffic. For Windows traffic, record a .pcap with Wireshark, copy it into pcaps/ and analyze it as above.

7.5 Health check

```
docker compose ps
curl -s 'localhost:9200/_cluster/health?pretty' | grep status
curl -s -o /dev/null -w "kibana: %{http_code}\n" localhost:5601/api/status
curl -s -o /dev/null -w "jupyter: %{http_code}\n" localhost:8888
```

Expected: all containers Up, status green, kibana 200, jupyter 302.

8. Viewing results in a GUI (no command line)

8.1 Kibana: http://localhost:5601 (open in Chrome on Windows)

- Menu -> Stack Management -> Data Views -> Create data view. Name: **cyberai-***, timestamp field: **@timestamp**, then Save.
- Menu -> Discover. Set the time range (top right) to Last 24 hours or Last 7 days.
- Search `tags: suricata and event_type: alert` to see Suricata alerts.
- Search `tags: zeek and conn_state: *` to see connections; click a field on the left (id.resp_h, conn_state) to see top values.
- Dashboard: Menu -> Dashboards -> Create dashboard -> Create visualization. Suggested panels: alerts over time, top destination IPs, conn_state bar chart, top server names (ssl).

8.2 Jupyter: http://localhost:8888/lab?token=...

Get the token with `docker compose logs jupyter | grep token`. Create a Python 3 notebook and run code in cells. Commands starting with % (such as %pip install) work only inside notebook cells, never in the Kali terminal.

8.3 Wireshark (raw packets)

Install Wireshark in Windows and open the capture from this Windows path:

```
\\wsl.localhost\kali-linux\home\bhosalepb01\cyberai-lab\pcaps
```

Useful filters: `http`, `tls`, `tcp.flags.syn==1 && tcp.flags.ack==0` (connection attempts), `ip.addr==169.254.169.254`.

Zeek and Suricata have no GUI of their own; Kibana is the GUI for both.

9. Understanding the logs

9.1 Zeek logs (~/cyberai-lab/logs/zeek/)

File	Contents
conn.log	Every connection: IPs, ports, protocol, duration, bytes, packets, conn_state, history
ssl.log	HTTPS sessions with server_name (e.g. telemetry.elastic.co)
http.log	Plain HTTP requests: host, URI, method, user-agent, status
dns.log	DNS lookups (absent in this capture)
files.log	Files seen in transfers, with hashes and MIME types
packet_filter.log	Zeek's capture filter information

conn_state	Meaning
SF	Normal: handshake and clean close
S0	SYN sent, no reply (failed attempt, blocked, or a scan if many different hosts/ports)
S1	Established but not cleanly closed
REJ	Connection rejected by the target
RSTO / RSTR	Reset by the originator / by the responder

9.2 Suricata logs (~/cyberai-lab/logs/suricata/)

File	Contents
eve.json	All events as JSON; event_type is alert, flow, http, tls, dns, fileinfo or stats. Logstash reads this file.
fast.log	One human-readable line per alert: time, [gid:sid:rev], signature, classification, priority, src -> dst
stats.log	Engine performance counters
suricata.log	Engine messages and errors

Filter on event_type when analyzing: most rows in eve.json are periodic 'stats' records, not network events.

9.3 What the first results showed

Traffic	Explanation
HTTPS to telemetry.elastic.co (9), artifacts.security.elastic.co (2)	Elasticsearch/Kibana containers sending telemetry and fetching rule artifacts. This is the lab's own baseline traffic.
8 connections in state S0 to 169.254.169.254:80 and 34.223.124.45:80	SYN packets with no reply. 169.254.169.254 is the cloud-metadata address that software probes to detect a cloud VM; nothing answers on a laptop. Benign.
1 real HTTP request to 104.20.23.154:80	The curl to example.com. Triggered the custom rule sid 1000001.
254 'SURICATA TCPv4 invalid checksum' alerts (first run)	False positives from WSL checksum offloading. Fixed with the -k none option.

10. Suricata IDS and IPS

	IDS (what this project runs)	IPS
Position	Watches a copy of traffic (passive)	Inline: traffic passes through it
Action	alert (detect and log only)	drop / reject (block)
Needs	A capture interface or PCAP file	A gateway/firewall host with two interfaces (NFQUEUE or AF_PACKET inline)
On this PC	Works: PCAP analysis and live capture inside WSL	Not practical in WSL. Practice in a Linux VM with two network adapters (VirtualBox/VMware).

Useful commands

```
sudo suricata-update                                    # update rules (about weekly)
sudo suricata -T -c /etc/suricata/suricata.yaml -S ~/cyberai-lab/local.rules   # test rules
cat ~/cyberai-lab/logs/suricata/fast.log                # read alerts
grep -o '"signature":"[^"]*"' ~/cyberai-lab/logs/suricata/eve.json | sort | uniq -c
```

Tuning ideas for the report: suppress noisy rules, add rules for suspicious behavior (many S0 attempts to different ports = scan), and compare alert counts before and after tuning.

11. Machine learning in the lab

11.1 K-Means (done)

Unsupervised: groups similar connections using duration, bytes, packets and port after scaling. Results on 20 connections:

Cluster	Size	Profile
0	10	HTTPS (443) to Elastic servers, about 1-2 KB each way, about 6 s: telemetry baseline
1	9	Port 80; 8 were empty failed attempts (S0) plus 1 real HTTP request
2	1	HTTPS with about 68 KB received: the outlier (largest download)

Lesson learned: the script filled missing values with 0, which put failed connections next to a real request. Missing values and true zeros are different; drop missing rows or keep conn_state as a feature. Improved code:

```
num = df[cols].apply(pd.to_numeric, errors="coerce").dropna()
df = df.loc[num.index]
```

Run: `docker compose exec jupyter python work/kmeans.py`. With only about 20 connections this is a demonstration, not a reliable model.

11.2 Logistic Regression (to do)

Supervised binary classification: normal (0) vs attack (1). Your own captures have no labels, so use a labeled dataset such as NSL-KDD, UNSW-NB15 or CIC-IDS2017 (or the file your course provides). Put the CSV in ~/cyberai-lab/notebooks/ (it appears as work/ in Jupyter). Template:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

data = pd.read_csv("work/your_dataset.csv")
y = (data["label"] != "normal").astype(int)         # adjust the label column/name
X = data.drop(columns=["label"]).select_dtypes("number")

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
sc = StandardScaler().fit(Xtr)
model = LogisticRegression(max_iter=1000).fit(sc.transform(Xtr), ytr)
pred = model.predict(sc.transform(Xte))
print(classification_report(yte, pred))
print(confusion_matrix(yte, pred))
```

Metric	Meaning in security terms
Precision	Of the connections flagged as attacks, how many really were (low = many false alarms)
Recall	Of the real attacks, how many were caught (low = missed attacks)
F1	Balance of precision and recall
Confusion matrix	True/false positives and negatives; shows the type of mistakes

12. MISP (to do)

MISP wouldn't start earlier (misp-core unhealthy) because memory was short. Retry it alone:

```
cd ~/cyberai-lab && docker compose down        # free RAM first
cd ~/misp-docker && docker compose up -d
docker compose ps
docker compose logs misp-core | tail -40       # if still unhealthy
```

Open https://localhost (accept the self-signed certificate warning). Change the default admin password in .env before use. Then enable feeds under Sync Actions -> Feeds (for example CIRCL OSINT) and look at events and attributes (IPs, domains, hashes). Optional integration: compare Zeek destination IPs and domains with MISP indicators in a notebook.

13. Troubleshooting guide

Symptom	Cause and fix
Docker Desktop stuck on 'Engine starting'	WSL backend problem on ARM. Project uses Docker Engine inside Kali instead (sudo service docker start).
Elasticsearch exits immediately	sudo sysctl -w vm.max_map_count=262144 (resets after wsl --shutdown)
'address already in use' on port 9200	A native service holds it: sudo ss -ltnp | grep 9200, stop that service (elasticsearch, wazuh-indexer)
'no configuration file provided'	docker-compose.yml was pasted into the terminal instead of saved. Use cat > file <<'EOF' ... EOF
zeek: command not found	Add /opt/zeek/bin to PATH (section 5.2)
%pip: fg: job not found	% commands work only in Jupyter cells. Use docker compose exec jupyter pip install ...
Python pasted in terminal gives zsh errors	Python belongs in .py files or notebook cells
curl -X DELETE 'cyberai-*' is refused	Elasticsearch 8 blocks wildcard deletes. Use the exact index name.

Symptom	Cause and fix
Logstash: object mapping for [host/http_host]	Field-type clash. Use the ruby filter (6.2), delete the index, restart Logstash.
Kibana Discover empty	Widen the time range, check data view cyberai-*, check docs.count in _cat/indices
Tool is slow or WSL runs out of memory	Run fewer containers; never run MISP and ELK together on limited RAM; check free -h
Suricata unix-socket warning	Harmless in PCAP/offline and live test runs

14. What should be done next (action plan)

#	Task	How to know it is done
1	Create the Kibana data view cyberai-* and open Discover	Zeek and Suricata events visible with a time range
2	Build a Kibana dashboard (alerts over time, top destinations, conn_state, server names)	Dashboard saved; screenshot taken
3	Confirm live Suricata alerts reach Kibana	New LAB alert appears within about 30 s of a curl to example.com
4	Generate more varied traffic and analyze it (separate Zeek folder per run)	At least 100 connections in Elasticsearch
5	Re-run K-Means with the dropna fix; try k=2,3,4	Clusters explained; outliers examined
6	Run Logistic Regression on a labeled dataset	Classification report and confusion matrix produced
7	Compare both models and discuss results	Short written comparison: supervised vs unsupervised
8	Run MISP, enable one feed, review indicators	MISP login works; events visible
9	Write one extra Suricata rule and test it	Rule validated with -T and fires on test traffic
10	Open the PCAP in Wireshark and match it to conn.log	Can explain one S0 and one SF connection
11	Write the final report and save backups	Report, screenshots and configs stored

Suggested deliverables for submission

- Architecture diagram and short explanation of each tool (sections 1-2).
- Screenshots: docker compose ps, Kibana Discover, the dashboard, a Suricata alert, Jupyter output (clusters and classification report).
- Configuration files: docker-compose.yml, pipeline.conf, local.rules, the Python scripts.
- Results discussion: baseline traffic, S0 attempts, cluster meanings, model metrics, limitations.
- Lessons learned: ARM compatibility, port conflicts, Logstash field clash, checksum false positives.

Backup and shutdown

```
cd ~/cyberai-lab && docker compose down        # data stays in the esdata volume
tar czf ~/cyberai-backup.tgz ~/cyberai-lab/{docker-compose.yml,logstash,local.rules,notebooks,pcaps}
cp ~/cyberai-backup.tgz /mnt/c/Users/'PRATIK BHOSLE'/Documents/
# in PowerShell, to free all RAM:  wsl --shutdown
```

Limitations to mention: the lab runs on one laptop with a small, self-generated dataset; live capture sees only WSL traffic; Elasticsearch security is disabled; true IPS needs an inline setup; K-Means on about 20 connections is only a demonstration.

Glossary

Term	Meaning
PCAP	File with captured network packets
IDS / IPS	Intrusion Detection / Prevention System
ELK	Elasticsearch, Logstash, Kibana
IOC	Indicator of compromise (malicious IP, domain, hash)
SID	Suricata rule signature ID
K-Means	Unsupervised clustering into k groups
Logistic Regression	Supervised classifier that outputs the probability of a class